A clear starting point for the AuditFlow UI

A compact handoff for building the right frontend - grounded in the repository as it exists today.

PROJECT HANDOFF | NOVEMBER 2026

AuditFlow is intended to help a user submit a document, follow its processing, review extracted or analyzed output, and archive a completed record. Repository names point toward this workflow: UploadZone, Dashboard, extract, analyze, and archive.

What success looks like

- A user can tell what was submitted, what is happening, what the system found, and what action comes next.
- The interface feels like a clear, professional admin dashboard and works on desktop and mobile.
- Real service behavior is connected only after its API and business rules are confirmed.

Confirmed technical context

- Frontend stack: React 19, TypeScript, and Vite.
- The current App.tsx still renders the Vite starter page.
- Dashboard.tsx and UploadZone.tsx exist but are empty.
- Backend function files (extract, analyze, archive) and backend/template.yaml are empty.
- No endpoints, data schema, file rules, authentication, or design system are documented.

> **Important: scaffold, not working product**
> The intended workflow below is a proposed direction inferred from repository names. Confirm requirements with the project owner. Do not treat example UI states or service contracts as implemented backend behavior.

Frontend objective

Replace the starter screen with a responsive document dashboard and a complete upload-to-review experience. Keep the integration boundary typed and easy to connect later; do not make up API details or show fake success.

Suggested user journey

1 Choose document > 2 Upload > 3 Process > 4 Review > 5 Archive

Show a clear status at every stage. A success state must only appear after the service confirms success.

Core screens and interactions

Dashboard

- Product name, clear page heading, and prominent Upload document action.
- Recent document/audit records; show only fields actually supplied by the backend.
- Helpful first-use empty state, plus distinct loading and error states.

Document upload

- Accessible drop area and file picker; show selected filename and remove/replace action.
- Validate only file types and size limits confirmed by the product/backend owner.
- Show upload or processing progress when real progress/status is available.

Review results

- Show document identity and current processing status.
- Present returned extraction/analysis fields in a readable, structured layout.
- Handle missing values and service errors; offer retry only where supported.

Archive

- Make the action explicit; confirm before an irreversible change.
- Reflect only a server-confirmed result; explain failures and allow a safe retry.

States the interface should represent

Empty | Selecting | Uploading | Processing | Completed | Failed / retry | Archiving | Archived

> **Design direction**
> Professional, calm admin UI with strong status hierarchy, readable cards/tables, responsive layouts, keyboard-accessible controls, visible focus, and clear errors. No approved brand colors or design reference exist; keep styling tokens easy to change.

Responsive behavior

On desktop, use a compact header/navigation with a full-width records area. On mobile, stack content, keep primary actions reachable, and use cards rather than forcing wide tables to scroll.

Suggested component boundaries

App / page shell

Page composition, navigation and shared layout. Keep global state minimal and typed.

Dashboard

Records list, useful empty/loading/error variants, and selected-record navigation. Add search/filter only if supported by requirements.

UploadZone

File selection and drag/drop, client-side presentation state, validation against confirmed rules, and upload callbacks.

Record detail / results

Status, returned extraction/analysis data, retry/archive actions, and explicit failure handling.

Integration rules

- Put API calls behind a small typed service layer (or typed props) so endpoint details can be supplied later.
- Use an environment variable for the API base URL after it is known; do not hardcode a guessed URL.
- Do not invent backend fields, status values, auth, file constraints, or success responses. Keep development mocks isolated and clearly labeled.
- Represent loading, success, and failure distinctly. Surface errors; do not silently turn failures into success-shaped data.
- Avoid adding libraries unless approved or an existing dependency fits.

Questions to resolve before live API wiring

Files

Which file types and size limits are allowed? One or multiple files? Where are uploads sent?

Workflow

What are the exact statuses? Is progress polled or pushed? What are retry rules and archive semantics?

Data and access

What are the endpoints, request/response shapes, authentication rules, result fields, and retention requirements?

> **Confirmed stack**
>
> React 19 + TypeScript + Vite. Preserve the existing package setup unless the implementation genuinely requires a change.

Prompt to send to a frontend developer

Share this PDF together with the repository so the implementer has both product direction and source context.

Build the frontend for AuditFlow in the existing repository. It is intended to be a document audit/processing app: a user selects and uploads a document, follows extraction/analysis status, reviews the returned result, and can archive a completed record.

Use the existing stack and conventions: React 19, TypeScript, and Vite. Inspect the source first. Replace the Vite starter screen with a responsive, polished admin-style experience: dashboard, document upload area (drag/drop and file picker), record/status list, result-review area, and archive confirmation. Include accessible labels and keyboard focus. Clearly represent empty, loading, uploading, processing, completed, failed/retry, and archived states.

Important repository constraint: backend function files and infrastructure template are empty. There are no confirmed API endpoints, auth rules, file limits, status enum, or request/response schemas. Do not invent these or show fake success. Keep integration behind a small typed service boundary with TODOs. Use mock data only if it is isolated and unmistakably development-only. Ask for the missing contract before connecting real endpoints.

Keep components small and typed, handle errors explicitly, avoid unnecessary dependencies, and make the layout usable on mobile and desktop. Update directly related docs if needed. Add focused tests for meaningful behavior introduced. Run the existing frontend build and lint scripts. Report changed files, assumptions, and remaining API questions.

Acceptance checklist

- No Vite starter content; dashboard and upload views reflect the AuditFlow workflow.
- Key states are visible and interactive, with no silent failure or hardcoded success.
- Mobile/desktop layouts are usable; controls are accessible.
- API details remain unresolved until supplied; UI can connect without rewriting page components.
- Existing build/lint checks pass; implementer reports remaining contract questions.

Bottom line

This brief explains the intended product flow. The repository is the source of truth for the implementation; backend contracts still need to be provided for a truly connected frontend.